Федеральное агентство по рыболовству

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Астраханский государственный технический университет»

Система менеджмента качества в области образования, воспитания, науки и инноваций

ООО «ДКС РУС» по международному стандарту ISO 9001:2015

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И КОММУНИКАЦИЙ

КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЗАЩИТА ИНФОРМАЦИИ

ОТЧЕТ

о выполнении лабораторной работы № 4

Реализация электронной цифровой подписи

Выполнил:
студент гр. ЗИНРб-51

_____ Генералов В. В.

«_____» _____ 2026 г.

Преподаватель:

_____ Савоськин И. А.

«_____» _____ 2026 г.

Астрахань – 2026

ВВЕДЕНИЕ

Электронная цифровая подпись (ЭЦП) удостоверяет авторство и неизменность электронного документа. В работе ЭЦП строится из программ предыдущих лабораторных работ: алгоритма шифрования RSA (работа № 2) и собственной хэш-функции (работа № 3).

Цель работы: «ознакомиться с принципами формирования и проверки электронной цифровой подписи. Получить практические навыки программирования ЭЦП с использованием алгоритма RSA и хэш-функции».

Задание по методическим указаниям состоит из четырёх пунктов:

1. «С помощью программ, выполненных в предыдущих лабораторных, реализовать и запрограммировать алгоритм электронной цифровой подписи с помощью алгоритмов RSA и самостоятельно придуманной хэш-функции».
2. «Сообщением, подписанным ЭЦП должен быть текст, поддерживающий все символы ASCII».
3. «Программа должна сама подписывать сообщение, а также проверять подпись».
4. «Добавить в программу возможность модифицировать передаваемое подписанное сообщение, тем самым нарушая его целостность. На этапе проверки подписи такого сообщения программа должна найти несоответствие хэш-сумм».

Вариант хэш-функции – 1: хэш-функция работы № 3 с основной функцией sin(***m***) и выходом 32 бита.

В работе разработана программа, которая подписывает текстовое сообщение, проверяет подпись и демонстрирует обнаружение модификации подписанного сообщения. Выполнение пунктов задания подтверждено экранами режимов программы, таблицей модификаций подписанного сообщения и автоматическим тестом на 1000 случайных сообщений.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

ЭЦП – число S, вычисленное из сообщения M закрытым ключом отправителя и проверяемое его открытым ключом. ЭЦП предназначена для обеспечения аутентичности (подтверждение, что сообщение подписано именно данным отправителем), целостности (невозможность изменения сообщения без обнаружения) и неотказуемости (отправитель не может отказаться от авторства).

Схема на основе RSA использует ключи из работы № 2: модуль $n = p \cdot q$, где p и q – простые числа, функцию Эйлера $\varphi = (p - 1)(q - 1)$, закрытый ключ d и открытый ключ e (вместе с модулем – пары $\{d, n\}$ и $\{e, n\}$), связанные условием $e \cdot d \equiv 1 \pmod{\varphi}$. Отправитель подписывает хэш-сумму сообщения закрытым ключом и передаёт пару (M, S):

$$h = H(M),\ S = h^d \bmod n, \tag{1.1}$$

где

h – хэш-сумма сообщения;

H – хэш-функция;

M – сообщение;

S – подпись;

d – закрытый ключ;

n – модуль из алгоритма RSA.

Получатель вычисляет хэш-сумму сообщения и дешифрует подпись открытым ключом:

$$h' = H(M),\ h'' = S^e \bmod n, \tag{1.2}$$

где

h' – хэш-сумма полученного сообщения;

h'' – хэш-сумма, восстановленная из подписи;

e – открытый ключ.

Затем получатель сравнивает h' и h'': если они совпадают, подпись верна. Корректность схемы следует из $e \cdot d \equiv 1 \pmod{\varphi}$ и теоремы Эйлера при НОД$(h, n) = 1$ (для h, кратных p или q, – из китайской теоремы об остатках): $S^e \equiv h^{d \cdot e} \equiv h \pmod{n}$, то есть $h'' = h \bmod n$. Отсюда условие применимости схемы $h < n$: иначе восстанавливается не h, а его остаток по модулю n.

Подписывается не сообщение, а его хэш-сумма фиксированной длины, которая укладывается в один блок RSA. Стойкость RSA-части определяется трудностью факторизации n, стойкость к подмене – односторонностью и стойкостью к коллизиям хэш-функции.

2 ОПИСАНИЕ АЛГОРИТМА И ПРОГРАММЫ

2.1 Постановка задачи

По заданию программа должна подписывать текст ASCII по формуле (1.1), проверять подпись по формуле (1.2) и модифицировать подписанное сообщение с последующей проверкой. Методические указания не определяют ряд деталей, поэтому приняты следующие допущения:

1. Хэш-функция – ***my_hash*** из работы № 3, вариант sin(***m***), выход 32 бита; алгоритм RSA – из работы № 2 (простые ***p*** и ***q***, выбираемый закрытый ключ ***d***, открытый ключ $e = d^{-1} \bmod \varphi$).
2. Условие $h < n$ обеспечивается тем, что простые числа ***p*** и ***q*** берутся из диапазона $[2^{17}, 2^{18})$: тогда $n \geq 2^{34} > 2^{32} > h$; ключи примера из методических указаний к работе № 2 ($n = 33$) для ЭЦП непригодны, что показывает отдельный режим.
3. Ключи проверяются на простоту ***p*** и ***q*** и условия $p \neq q$, $p \cdot q > 2^{32}$, $1 < d < \varphi$, НОД(d, φ) = 1, что обеспечивает существование ***e*** и корректность проверки.
4. Сообщение – строка любой длины из символов с кодами 0–127, включая пустую (вводится одной строкой); символы вне ASCII отклоняются с повтором ввода.
5. Модификация – модель злоумышленника в канале связи: замена, вставка, удаление символа или перестановка соседних символов, вручную или случайно; подпись ***S*** при этом не пересчитывается.
6. Проверка выполняется для текущей пары (***M***, ***S***), для изменённого сообщения с прежней подписью ***S*** и для пары, введённой вручную; при $h'' \geq 2^{32}$ значение ***S*** не является подписью под данным ключом, так как 32-битная хэш-сумма не может принять такое значение.
7. Правильность подтверждается автоматическим тестом: 1000 случайных сообщений длиной от 0 до 50 символов должны проходить проверку нетронутыми и отклоняться после модификации.

2.2 Описание алгоритма

Подпись формируется по формуле (1.1): хэш-сумму ***h*** вычисляет функция ***my_hash*** работы № 3, возведение в степень по модулю – функция быстрого возведения в степень ***modpow*** работы № 2. Проверка выполняется по формуле (1.2) со сравнением h' и h''.

Обнаружение модификации. Изменённое сообщение M' даёт $h' = H(M')$, тогда как $h'' = H(M)$ восстанавливается из неизменной подписи ***S***. Подпись сходится только при коллизии $H(M') = H(M)$; для 32-битной хэш-суммы вероятность этого при случайном изменении равна 2^{-32}, а число различающихся бит h' и h'' в среднем близко к 16 (лавинный эффект работы № 3).

Псевдокод основных функций программы приведён на рисунке 2.1.

```
sign(M, d, n):                          verify(M, S, e, n):
    h ← my_hash(M)                          h′ ← my_hash(M)
    если h ≥ n: ошибка                      h″ ← modpow(S, e, n)
    S ← modpow(h, d, n)                     вернуть (h′, h″, h′ = h″)
    вернуть (h, S)

modify(M, вид, позиция, символ):        make_keys(p, q, d):
    replace: M[позиция] ← символ            p, q — простые из [2^17, 2^18) (или заданные)
    insert:  вставить символ                проверки: p ≠ q, p·q > 2^32, 1 < d < φ,
    delete:  удалить M[позиция]                 НОД(d, φ) = 1
    swap:    M[i] ↔ M[i+1]                  e ← d^(-1) mod φ (расширенный алгоритм Евклида)
    S не пересчитывается                    вернуть {e, n}, {d, n}
```

Рисунок 2.1 – Псевдокод основных функций

2.3 Описание программы

Программа написана на языке Python 3 с использованием только стандартной библиотеки и запускается из консоли без аргументов. Из предыдущих работ подключаются модуль шифрования RSA работы № 2 (возведение в степень по модулю, проверка простоты, генерация простых чисел, ключи, ввод чисел и текста ASCII) и модуль хэш-функции работы № 3 на основе sin(***m***) (вычисление хэш-суммы, шестнадцатеричный вывод, расстояние Хэмминга).

Ядро программы составляют функции ***sign***, ***verify*** (возвращает ***h′***, ***h″*** и результат сравнения), ***modify*** (четыре вида модификации без пересчёта ***S***), ***make_keys*** (формирование ключей с проверками) и ***self_test*** (автоматический тест, режим 6). Состояние сеанса – ключи, сообщение, подпись, изменённое сообщение – хранит класс ***State***; остальные режимы меню выполняют функции с префиксом ***mode_***.

Главное меню программы содержит семь режимов и команду выхода «0». Режимы 1 и 2 задают ключи RSA (случайные или введённые вручную) и сообщение (с клавиатуры или случайное из печатных символов ASCII); режим 3 подписывает сообщение и выводит ***h***, ***S*** и пару (***M***, ***S***); режим 4 проверяет текущую пару, изменённое сообщение с той же подписью ***S*** или введённые ***M*** и ***S***; режим 5 изменяет подписанное сообщение и проверяет его; режим 6 запускает автоматический тест; режим 7 демонстрирует условие $h < n$ на ключах $n = 33$ из примера методических указаний к работе № 2. Режимы 3–5 без ключей, режимы 3 и 5 без сообщения и режим 5 без подписи выводят подсказку, а режим 4 при отсутствии подписи сразу запрашивает ***M*** и ***S***. Неверный ввод (ответ вне вариантов подменю, не простое ***p*** или $p > 10^{12}$, $p \cdot q \le 2^{32}$, ***d*** вне $(1, \varphi)$ или не взаимно простое с φ, символ вне ASCII, ***S*** вне $[0, n-1]$, вид модификации, невозможный для данного сообщения) отклоняется с указанием причины и повтором запроса; «Ctrl+C» и «Ctrl+D» завершают программу.

3 РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

3.1 Ключи RSA и сообщение ASCII

Ключи для экранов режимов 1–5 заданы вручную (главное меню и режим 1 показаны на рисунке 3.1): $p = 131101$, $q = 262139$, $d = 1234567$; отклонены $q = p$ и пара $p = 61$, $q = 53$ из работы № 2 ($n = 3233 < 2^{32}$). Пример случайных ключей: $p = 213029$, $q = 186859$, $n = 39806385911$.

```
Лабораторная работа №4. ЭЦП на RSA (ЛР2) и хэш-функции sin(m) (ЛР3)
  1 — ключи RSA (случайные или вручную)
  2 — сообщение (с клавиатуры или случайное)
  3 — подписать сообщение
  4 — проверить подпись
  5 — изменить подписанное сообщение и проверить
  6 — автотест на 1000 сообщений
  7 — демонстрация условия h < n (ключи n = 33 из примера ЛР2)
  0 — выход
Выбор: 1
  Ключи: 1 — случайные (p, q из [2^17, 2^18)), 2 — ввести p, q, d вручную: 2
  p (простое, p ≤ 10^12): 131101
  q (простое, q ≠ p, p·q > 2^32, q ≤ 10^12): 131101
  q должно отличаться от p: при p = q формула φ = (p-1)(q-1) неверна.
  q (простое, q ≠ p, p·q > 2^32, q ≤ 10^12): 262139
  n = 34366685039, φ = 34366291800
  d (1 < d < φ, НОД(d, φ) = 1; Enter — случайное): 1234567
  p = 131101, q = 262139, n = p·q = 34366685039 (> 2^32 = 4294967296), φ = 34366291800
  Открытый ключ {e, n} = {8601127903, 34366685039}, закрытый {d, n} = {1234567, 34366685039}
```

Рисунок 3.1 – Главное меню и режим 1: ключи RSA вручную с проверкой $q \neq p$

Пункт 2 задания подтверждает режим 2, показанный на рисунке 3.2: текст с пробелами, знаками «$», «,», «#» и цифрами принят, символ «П» отклонён, случайное сообщение составлено из печатных символов ASCII.

```
Выбор: 2
  Сообщение: 1 — ввести с клавиатуры, 2 — случайное из печатных ASCII: 1
  Текст (ASCII, Enter — пустое): Привет
  Символ «П» (код 1055) вне кодировки ASCII, повторите ввод.
  Текст (ASCII, Enter — пустое): Pay 100$ to Bob, order #42
  M = 'Pay 100$ to Bob, order #42' (26 символов)
Выбор: 2
  Сообщение: 1 — ввести с клавиатуры, 2 — случайное из печатных ASCII: 2
  Длина (1..200): 20
  M = '>SE)0DAzkCS&<J:ElL2X' (20 символов)
```

Рисунок 3.2 – Режим 2: сообщение с клавиатуры (отказ при символе вне ASCII) и случайное

3.2 Формирование и проверка подписи

Пункты 1 и 3 задания подтверждают режимы 3 и 4, показанные на рисунке 3.3: хэш-сумма $h = 97bcf201$ меньше n, подпись $S = 17008916931$, проверка нетронутой пары сходится.

```
Выбор: 3
  h = H(M) = 97bcf201 = 2545742337 (h < n: да)
  S = h^d mod n = 17008916931
  Передаётся пара (M, S): ('Pay 100$ to Bob, order #42', 17008916931)
Выбор: 4
  Проверить: Enter — текущую пару (M, S), m — изменённое сообщение с той же S, i — ввести M и S:
  h'  = H(M)         = 97bcf201
  h'' = S^e mod n    = 97bcf201
  Подпись верна: хэш-суммы совпадают, сообщение подлинно и не изменено.
```

Рисунок 3.3 – Режимы 3–4: формирование подписи и проверка нетронутого сообщения

Проверка пары, введённой вручную (режим 4, ответ «i»), показана на рисунке 3.4: произвольное $S = 12345$ даёт $h'' = 20006972960$, что больше 2^{32} и невозможно для 32-битной хэш-суммы. Подпись, искажённая в одном разряде ($S + 1$), при такой же проверке даёт $h'' = 6072391386$ с тем же вердиктом.

```
Выбор: 4
  Проверить: Enter — текущую пару (M, S), m — изменённое сообщение с той же S, i — ввести M и S: i
  M (ASCII): Pay 100$ to Bob, order #42
  S: 12345
  h'  = H(M)         = 97bcf201
  h'' = S^e mod n    = 20006972960 — не помещается в 32 бита: S не является подписью под этим ключом
  ПОДПИСЬ НЕВЕРНА: хэш-суммы не совпадают — подпись чужая или искажена.
```

Рисунок 3.4 – Режим 4: проверка пары (M, S) с чужой подписью

3.3 Модификация сообщения и автоматический тест

Пункт 4 задания подтверждает режим 5, показанный на рисунке 3.5: замена «1» на «9» превращает сумму 100$ в 900$, подпись не пересчитывается, и проверка находит несоответствие хэш-сумм. То же несоответствие выводит режим 4 при ответе «m».

```
Выбор: 5
  Модификация: 1 — заменить символ, 2 — вставить, 3 — удалить, 4 — переставить соседние, Enter —
      случайная: 1
  Позиция (0..25, Enter — случайная): 4
  Символ ASCII (Enter — случайный): 9
  Было:  'Pay 100$ to Bob, order #42'
  Стало: 'Pay 900$ to Bob, order #42'  — замена символа '1' на '9' в позиции 4; подпись S =
      17008916931 не менялась
  Проверка изменённого сообщения: h' = 4cc35b97, h'' = 97bcf201 → НЕ СОВПАДАЮТ (21 бит из 32) —
      целостность нарушена
Выбор: 4
  Проверить: Enter — текущую пару (M, S), m — изменённое сообщение с той же S, i — ввести M и S: m
  h'  = H(M)         = 4cc35b97
  h'' = S^e mod n    = 97bcf201
  ПОДПИСЬ НЕВЕРНА: хэш-суммы не совпадают (различаются 21 бит из 32) — сообщение изменено или
      подписано другим ключом.
```

Рисунок 3.5 – Режим 5: замена символа и проверка изменённого сообщения

Все модификации сообщения при той же подписи, включая замену символа (см. рис. 3.5), приведены в таблице 3.1: каждая обнаружена, подпись признана неверной.

Таблица 3.1 – Модификации подписанного сообщения при $S = 17008916931$, $h'' = 97bcf201$

Модификация	Сообщение после модификации	Хэш-сумма $h' = H(M')$	Различие h' и h'', бит
Замена «1» → «9», позиция 4	Pay 900$ to Bob, order #42	4cc35b97	21
Вставка «0» в позицию 7	Pay 1000$ to Bob, order #42	dd516147	16
Удаление «1» из позиции 4	Pay 00$ to Bob, order #42	31017195	16
Перестановка «4» и «2», позиции 24–25	Pay 100$ to Bob, order #24	dc2cfe6d	12
Случайная: « » → «5», позиция 3	Pay5100$ to Bob, order #42	6b5f035c	21

Автоматический тест (режим 6) на случайных ключах: 1000 нетронутых сообщений

длиной 0–50 символов из печатных ASCII прошли проверку, все 1000 изменённых отклонены.

Режим 7 показывает, почему нужны большие p и q: на ключах примера из методических указаний к работе № 2 ($n = 33$) хэш-сумма $h = 3264597241$ не меньше n, и проверка нетронутого сообщения не сходится – восстанавливается лишь $h \bmod 33 = 4$. Оба режима показаны на рисунке 3.6.

```
Выбор: 6
  Ключи: p = 241333, q = 251939, n = 60801194687 (> 2^32: да), e = 32935383707, d = 36875377235
  Сообщений: 1000 (длина 0..50, все печатные ASCII)
  Нетронутые прошли проверку: 1000/1000
  Изменённые отклонены:      1000/1000
Выбор: 7
  Ключи примера ЛР2: p = 3, q = 11, n = 33, e = 7, d = 3; сообщение 'Hello'
  h = 3264597241 (32 бита) ≥ n = 33; S = h^d mod n = 31; S^e mod n = 4 = h mod n = 4
  Проверка нетронутого сообщения: НЕ сошлась — при h ≥ n восстанавливается лишь h mod n.
  Поэтому p, q берутся из [2^17, 2^18): n ≥ 2^34 > 2^32 ≥ h.
```

Рисунок 3.6 – Режимы 6–7: автоматический тест на 1000 сообщений и условие $h < n$ на ключах $n = 33$

3.4 Анализ результатов

Обнаружение модификации. Все пять модификаций из таблицы 3.1 и 1000 случайных модификаций автоматического теста обнаружены. Хэш-суммы различаются на 12–21 бит, в среднем на 17,2 бита, что согласуется с лавинным эффектом хэш-функции работы № 3 (около 16 бит из 32). Модификация останется незамеченной только при коллизии $H(M') = H(M)$: вероятность этого для случайного изменения равна $2^{-32} \approx 2{,}3 \cdot 10^{-10}$, направленный поиск второго прообраза требует около 2^{32} вычислений хэш-функции.

Чужая подпись. Для S, не являющейся подписью, значение $h'' = S^e \bmod n$ распределено по всему диапазону $[0, n - 1]$; при $n = 34366685039$ оно меньше 2^{32} лишь с вероятностью $2^{32}/n \approx 0{,}125$, и даже тогда должно совпасть с h'. Оба примера подраздела 3.2 ($S = 12345$ и $S + 1$) дали $h'' > 2^{32}$, что программа отмечает отдельно.

Условие $h < n$. Результаты режимов 6 и 7 (см. рис. 3.6) подтверждают вывод раздела 1: при $n = 33$ проверка не сходится даже на нетронутом сообщении, при $n \geq 2^{34}$ сходится всегда (1000 из 1000). Диапазон $[2^{17}, 2^{18})$ взят с запасом: $n \geq 2^{34}$, тогда как 32-битной хэш-сумме достаточно любых простых с $p \cdot q > 2^{32}$ – это условие и проверяется при ручном вводе.

Таким образом, пункт 1 задания подтверждают режимы 1, 3 и 4 (см. рис. 3.1, 3.3), пункт 2 – режим 2 (см. рис. 3.2) и таблица 3.1, пункт 3 – режимы 3, 4 и 6 (см. рис. 3.3, 3.4, 3.6), пункт 4 – режимы 5 и 6 (см. рис. 3.5, 3.6) и таблица 3.1.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Для чего используется ЭЦП? Какие свойства имеет подписанное сообщение? ЭЦП удостоверяет авторство и неизменность электронного документа: договоров, платёжных поручений, обновлений программ, сертификатов. Подписанное сообщение обладает аутентичностью (подпись $S = h^d \bmod n$ мог вычислить только владелец d), целостностью (изменение M меняет h', и проверка не сходится, см. таблицу 3.1) и неотказуемостью (автор не может отрицать подпись, проверяемую его открытым ключом). Конфиденциальности подпись не даёт: сообщение M передаётся открыто.

Вопрос 2. Каким образом ЭЦП выявляет модификацию подписанного сообщения? Проверяющий вычисляет $h' = H(M)$ по полученному сообщению и $h'' = S^e \bmod n$ по подписи; h'' равно хэш-сумме исходного сообщения, h' – хэш-сумме полученного. Любое изменение M из-за лавинного эффекта меняет около половины бит хэш-суммы, и $h' \neq h''$ (совпадение возможно лишь при коллизии хэш-функции с вероятностью 2^{-32}): замена одной цифры изменила 21 бит из 32 (см. рис. 3.5). Подделать подпись под новое сообщение нельзя без d, а подобрать сообщение с прежней хэш-суммой – без поиска второго прообраза.

Вопрос 3. Почему в ЭЦП шифруется не само сообщение, а его хэш-сумма? Будет ли работать ЭЦП, если шифровать само сообщение? Хэш-сумма имеет фиксированную длину и заведомо меньше n: подпись – одно возведение в степень, её размер не зависит от длины документа. Подпись самого сообщения формально работает, если число M меньше n, но на практике непригодна. Длинный текст пришлось бы делить на блоки, как при шифровании в работе № 2, и подписывать каждый, поэтому подпись не короче документа (при посимвольных блоках работы № 2 примерно впятеро длиннее: число до $n \approx 2^{35}$ на 7-битный символ), время растёт пропорционально числу блоков, а блоки можно переставлять, удалять и дублировать без нарушения проверки (перестановка из таблицы 3.1 осталась бы незамеченной). Кроме того, RSA мультипликативна: из подписей M_1 и M_2 получается подпись $M_1 \cdot M_2 \bmod n$, а любое S даёт «подписанное» сообщение $S^e \bmod n$. Хэширование закрывает эти уязвимости.

Вопрос 4. Что такое сертификат ЭЦП? Из чего он состоит и для чего он требуется? Сертификат – электронный документ, связывающий открытый ключ с владельцем и заверенный подписью удостоверяющего центра (УЦ). По формату X.509 он содержит серийный номер, сведения о владельце, открытый ключ и алгоритм, наименование УЦ, срок действия, область применения и подпись УЦ. Без сертификата проверяющий не знает, чей ключ проверяет: злоумышленник подменяет открытый ключ $\{e, n\}$ на свой $\{e', n'\}$ (атака «человек посередине» из работы № 1) и выдаёт свои подписи за чужие. Доверие строится по цепочке от корневого ключа УЦ; в России сертификаты выдают аккредитованные УЦ по закону «Об электронной подписи».

Вопрос 5. Что такое радужные таблицы и как им противостоять? Радужная таблица – заранее вычисленная структура для обращения хэш-функции на ограниченном множестве входов (паролей): цепочки чередующихся хэширования и функций редукции, различных на каждом шаге цепочки (отсюда название), от которых хранятся лишь начало и конец; память обменивается на время поиска. Противостоят ей соль – случайная строка, уникальная для каждого пароля (таблицу пришлось бы строить для каждой соли заново; в работе № 3 соль задаётся аргументом ***salt***), медленные функции формирования ключа (PBKDF2, bcrypt, Argon2), секретный «перец» на стороне сервера и длинные пароли.

ЗАКЛЮЧЕНИЕ

В работе реализована электронная цифровая подпись на алгоритме RSA работы № 2 и хэш-функции варианта sin(***m***) работы № 3: подпись формируется по формуле (1.1), проверка выполняется по равенству $\boldsymbol{H}(\boldsymbol{M})$ и $\boldsymbol{S}^{\boldsymbol{e}} \bmod \boldsymbol{n}$ согласно формуле (1.2). Разработанная программа генерирует и принимает ключи с проверками, подписывает любой текст ASCII, проверяет пары ($\boldsymbol{M}$, $\boldsymbol{S}$), модифицирует подписанное сообщение четырьмя способами и обнаруживает изменение.

Пункты 1–4 задания выполнены: нетронутое сообщение проходит проверку, все модификации из таблицы 3.1 отклонены с расхождением хэш-сумм на 12–21 бит, 1000 случайных модификаций автоматического теста отклонены полностью (1000 из 1000). Тем самым цель работы достигнута.

Установлено условие применимости схемы $\boldsymbol{h} < \boldsymbol{n}$: для 32-битной хэш-суммы нужен модуль $\boldsymbol{n} = \boldsymbol{p} \cdot \boldsymbol{q} > 2^{32}$ (генератор берёт $\boldsymbol{p}$ и $\boldsymbol{q}$ из $[2^{17}, 2^{18})$, $\boldsymbol{n} \geq 2^{34}$); ключи примера с $\boldsymbol{n} = 33$ непригодны.

Схема обнаруживает случайные и преднамеренные изменения, но учебные параметры не дают криптографической стойкости: модуль $\boldsymbol{n} \approx 2^{35}$ факторизуется перебором делителей до $\sqrt{\boldsymbol{n}} \approx 185\,000$ за доли секунды (атака из работы № 2), после чего злоумышленник вычисляет $\boldsymbol{d}$ и подписывает любые сообщения; 32-битная хэш-сумма допускает поиск коллизии примерно за 2^{16} сообщений (работа № 3), то есть подготовку двух документов с одной подписью. Практические системы используют модуль 2048 бит и хэш-сумму 256 бит.